MPC-Based Drone Path Optimization And Obstacle Avoidance

Aayush Dulal
Department of Mechanical Engineering
The University of Texas at Dallas
Richardson, US
aayush.dulal@utdallas.edu

Bimlendra Ray
Department of Mechanical Engineering
Then University of Texas at Dallas
Richardson, US
bimlendra.ray@utdallas.edu

Caden B. Gigstad
Department of Mechanical Engineering
The University of Texas at Dallas
Richardson, US
cbg240000@utdallas.edu

Kaitlyn M. Oquinn
Department of Mechanical Engineering
The University of Texas at Dallas
Richardson, US
Kaitlyn.Oquinn@UTDallas.edu

***Abstract*—This project develops a nonlinear Model Predictive Control (MPC) framework for autonomous drone navigation in environments with multiple no-fly zones. Using a simplified dynamic model and a horizon-based optimization approach, the controller predicts future states and generates safe, energy-efficient trajectories while enforcing obstacle avoidance and control constraints. Simulation results in 2D demonstrate successful navigation through dense obstacle fields, smooth velocity regulation, and controlled energy consumption. An initial extension to 3D with cylindrical obstacles further shows the scalability of the approach and highlights opportunities for future refinement. Overall, the results confirm that nonlinear MPC is an effective method for constrained drone path planning.**

***Index Terms*—Drone, MPC, FMINCON, Optimization, Obstacle avoidance**

I. Introduction

When it comes to controlling dynamic systems, PID controllers and Linear Quadratic Regulators (LQRs) are the most common algorithms. PID controllers are well-suited for low-level control when a reference signal is available, and in the case of quadrotor control, a reference trajectory is typically generated using a trajectory generation algorithm. That is the same with LQR as well, for which we find the optimal gains for a linear system by optimizing for the design criteria needed. In addition to LQR only applying to linear systems, they also do not handle constraints directly and suffer from performance reduction due to uncertainties and disturbances. Hence, in contrast to these controllers, where a fixed control gain is used, model predictive controllers (MPC) provide a more dynamic and online option, which is very promising for controlling systems such as a quadrotor.

Model predictive controllers operate by solving an optimization problem at every time step to find the optimal control actions for multiple steps in the future. The horizon defines how many future steps the optimization problem is solved for, and the states of the system are sampled based on the predefined sampling time. This means that the optimal actions for the system are also changing at every sampling instant, and given uncertainties prevalent during real-world deployment, solving an optimization problem to find the optimal control is a very effective approach. This is especially evident when control of systems such as a quadrotor is performed.

Verification of its potential for applications such as quadrotor control was the major goal of this project. To make the experiments realistic, a simplified two-dimensional map of a residential block in Lakes of Coppell was created, which acted as the site of operation for the quadrotor. A two-dimensional quadrotor model was then used to perform a pick and drop task. The following sections will provide details on this and show the effectiveness of the non-linear MPC to generate an optimal trajectory and track it, resulting in successful delivery. For future work, a three-dimensional version of the experiment and the use of three-dimensional dynamics are also briefly discussed.

II. Related Work

Research on quadrotor control has evolved from linearized controllers to geometric and optimization-based methods to address the nonlinear and underactuated dynamics of the vehicle. A fundamental contribution is the geometric control framework of [1], which models the quadrotor directly on the configuration manifold SE(3), avoiding Euler-angle singularities and ensuring a smooth, globally valid attitude representation. Their rigid-body model enables accurate tracking under large rotational motions and forms the basis for many modern nonlinear controllers.

Building on geometric formulations, recent work has incorporated optimization and constraints through Nonlinear Model Predictive Control (NMPC). A representative example is the SE(3)-based NMPC architecture of [2], which integrates full quadrotor dynamics, augmented integral action, and an ellipsoidal obstacle-avoidance framework. This formulation enables aggressive maneuvers, such as navigating narrow gaps

and fast obstacle deviations, by explicitly considering attitude in the collision-check process.

In this project, the quadrotor model is used from the [1] for our dynamics, and obstacle avoidance is done by measuring the nominal distance. For optimization, the most commonly used framework, FMINCON, is used.

III. Quadrotor Dynamics and Model

A. 2D Planar Non-linear Quadrotor Dynamics

The exact quaternion-based 3D nonlinear quadrotor model is given in the paper [1]. For our project work, the dimension of the model is reduced to 2D, and the hovering condition is assumed to simplify the complexity of the problem. We consider a planar quadrotor moving in the x–y plane with pitch angle ψ. The state vector is

$$x = \begin{bmatrix} p_x \\ p_y \\ \psi \\ v_x \\ v_y \end{bmatrix},$$

where x, y denote position, ψ is the pitch angle, v_x, v_y are the corresponding velocities, and $\omega = \dot{\psi}$ is the angular rate. The control inputs are the acceleration a and the control angular speed ω.

The dynamics are:

$$\dot{x} = v_x, \tag{1}$$

$$\dot{y} = v_y, \tag{2}$$

$$\dot{\psi} = \omega, \tag{3}$$

$$\dot{v}_x = -\frac{F}{m}\sin\theta, \tag{4}$$

$$\dot{v}_y = \frac{F}{m}\cos\theta, \tag{5}$$

$$\tag{6}$$

B. 2D Quadrotor dynamics with energy percentage

The state vector is:

$$x = \begin{bmatrix} p_x \\ p_y \\ \psi \\ v_x \\ v_y \\ E \end{bmatrix},$$

The inputs of the system are defined as

$$a = u_1, \qquad \omega = u_2,$$

where a denotes the commanded translational acceleration and ω represents the commanded yaw rate. These two scalar inputs provide forward thrust and turning capability, capturing the essential actuation of a planar UAV or ground robot.

a) Position Kinematics.:

$$\dot{p}_x = v_x, \qquad \dot{p}_y = v_y.$$

These equations state that the time derivatives of the position coordinates correspond directly to the velocity components in the inertial frame. They describe purely kinematic relationships without involving the system's orientation.

b) Velocity Dynamics.:

$$\dot{v}_x = a\cos\psi - k_d v_x, \qquad \dot{v}_y = a\sin\psi - k_d v_y.$$

The forward acceleration command a acts along the body x-axis. Its contribution to the inertial-frame velocities is obtained by projecting it through the heading angle ψ. The terms $-k_d v_x$ and $-k_d v_y$ model aerodynamic drag or viscous damping, causing the velocity to decay proportionally to its magnitude.

c) Heading Dynamics.:

$$\dot{\psi} = \omega.$$

The vehicle's heading evolves directly according to the yaw-rate input. This simplified rotational model is common in unicycle and planar UAV dynamics, where orientation inertia is neglected.

d) Energy Dynamics.:

$$\dot{E} = -\alpha\,(a^2 + \omega^2).$$

The energy variable E decreases proportionally to the squared control inputs. This formulation penalizes aggressive maneuvers and captures the intuition that both forward acceleration and rapid turning consume additional energy. The positive constant α regulates the rate of depletion.

Overall, these equations form a reduced-order planar model suitable for trajectory planning and optimal control, while retaining essential nonlinearities such as orientation-dependent motion and energy-aware actuation.

IV. MPC Formulation

Model Predictive Control (MPC) is used to compute, at every time step, the optimal acceleration and yaw-rate inputs that drive the drone toward the target while respecting energy, dynamics, and obstacle constraints. The controller uses a prediction model of the drone and repeatedly solves a finite-horizon optimization problem to decide the best control action.

At each sampling instant, the MPC constructs a prediction of the drone's future motion over a horizon of $N_p = 15$ with sampling time $T_s = 0.2$ Using the current state as the initial condition, the model simulates forward N_p future states based on candidate control inputs. An optimization solver then selects the control sequence that minimizes a cost function while satisfying constraints.

Only the first control input of the optimized sequence is applied to the drone. At the next time step, the state is updated, the horizon shifts forward, and the entire optimization repeats.

The MPC balances multiple objectives through a quadratic cost, which penalizes the tracking error and energy use and also discourages unnecessary acceleration.

The nonlinear MPC problem with a terminal cost is

$$\min_{\{u_k\}_{k=0}^{N-1}} \quad J = \sum_{k=0}^{N-1} \left[(x_k - x_{\text{ref}})^\top Q (x_k - x_{\text{ref}}) + u_k^\top R u_k \right] + (x_N - x_{\text{ref}})^\top P (x_N - x_{\text{ref}})$$

$$\begin{aligned} \text{s.t.} \quad & x_{k+1} = x_k + dt\, f(x_k, u_k), \qquad k = 0, \ldots, N-1, \\ & a_{\min} \le a_k \le a_{\max}, \quad w_{\min} \le w_k \le w_{\max}, \\ & (p_x - x_{c,m})^2 + (p_y - y_{c,m})^2 \ge (r_m + \Delta r)^2, \\ & \qquad\qquad m = 1, \ldots, M, \quad k = 0, \ldots, N, \\ & x_0 = x_{\text{cur}} \end{aligned}$$

Here, the state is $x_k = [p_x,\, p_y,\, v_x,\, v_y,\, \psi,\, E]^\top$, where (p_x, p_y) denotes planar position, (v_x, v_y) are inertial-frame velocities, ψ is the heading angle, and E is the remaining battery energy. The control input is $u_k = [a_k,\, w_k]^\top$, with a_k the forward acceleration and w_k the yaw rate. The reference state x_{ref} specifies the desired target state, and x_N is the terminal state at the end of the horizon. Each obstacle i is defined by center (x_i, y_i), radius r_i, and safety buffer Δr. The function $f(\cdot)$ encodes the nonlinear drone dynamics, dt is the integration timestep, and N is the prediction horizon. The set $\{u_k\}_{k=0}^{N-1}$ forms the decision variables of the MPC problem, with $x_0 = x_{\text{cur}}$ the measured initial state.

V. Obstacle Avoidance

A realistic no-fly zone environment was created to test the MPC algorithm by translating the structures present on a neighborhood map into circular representations of buildings and houses. This approach ensures that the resulting path-planning problem reflects practical navigation challenges, such as those encountered in low-altitude UAV delivery.

The neighborhood map contained building footprints, street layouts, and larger structures such as schools and water towers. Each building footprint was simplified to a circle whose center and radius were selected to capture the approximate area occupied by the structure. Circular obstacles were used for three reasons: 1. Smoothness for Nonlinear Optimization - circles allow differentiable distance functions appropriate for FMINCON, reducing the likelihood of solver stagnation. 2. Computational Efficiency - Circular boundaries require only one inequality constraint per obstacle per prediction step, which is important given the large number of constraints. 3. Conservative Modeling - Buildings often have irregular shapes, and representing them as circles slightly over-approximates their footprint, providing a safety buffer. The neighborhood map (figure 1) was originally provided in pixel coordinates. For use in the MPC simulation, distances were scaled to meters and radii were chosen to reflect proportional sizes of buildings relative to the neighborhood block, ensuring the obstacle geometry preserved relative spacing from the original map. This ensured that the drones operating environment preserved the relative distances, sizes, and layout relationships of the real-world neighborhood.

Using a real map instead of random obstacles significantly

Fig. 1. Neighborhood of Lake Coppel

improves the validity of the MPC results. The drone must perform urban-scale navigation, not just avoid random obstacles. Narrow passages, intersections, and block layouts emerge naturally, which cannot be easily replicated with random generation. The algorithm is tested in structured clutter, which is a challenge in real UAV path planning. The obstacle layout introduces asymmetric and multi-path decision regions, essential for evaluating whether MPC males intelligent route selections This environment pushes the controller to reason about long-term consequences of heading changes and acceleration decisions, not just avoid isolated obstacles.

Once converted from the neighborhood map, obstacles were incorporated into the MPC as:

$$\text{No-fly zone } m: \qquad (p_x - x_{c,m})^2 + (p_y - y_{c,m})^2 \ge (r_m + \Delta r)^2$$

with $\Delta r = 0.3\,\text{m}$ as an added safety buffer. Because the map produced more than 50 structures, the MPC operated under a highly constrained environment, making sub-stepping essential for reliable collision checking. The final obstacle set reflects both real-world navigational complexity and mathematically tractable constraint geometry, making it a strong test for evaluating the nonlinear MPC problem.

VI. Simulation

The MPC framework described in earlier sections was implemented in MATLAB using a modular structure consisting of four parts: the main simulation loop, drone dynamics, cost, and nonlinear constraints. The `main-MPC-drone-2d.m` is the main simulation file, whereas `mpc-constraints-2D.m` encodes the constraints, `mpc-cost-2D.m` calculates the cost, and `drone2D-dynamics.m` defines the dynamics of the system. This section explains how the code was organized, how the components interact during the simulation, and highlights the numerical strategies required for reliable convergence in the presence of numerous no-fly zones.

The simulation begins by loading the initial state of the drone, defining the prediction horizon, and setting the solver limits. At each timestep, the code uses FMINCON to solve the finite-horizon MPC problem. The nonlinear program was solved using `fmincon` with the SQP algorithm, a maximum of 80 iterations, and tolerances set to ConstraintTolerance $= 10^{-6}$ and StepTolerance $= 10^{-8}$, with all intermediate output suppressed.

The optimizer searches for a sequence of accelerations and yaw-rate commands that minimize the cost function while respecting the drone dynamics and all obstacle constraints. Only the first control input from the optimized sequence is applied to the actual system. The state is then updated using the dynamics model, and the process repeats. This "receding-horizon" setup allows the controller to react to changing conditions, since a new optimization problem is solved every step. To help the solver converge faster, instead of starting from zeros each time, the previous optimal input sequence is shifted forward by one timestep and reused as the initial guess. This reduces computation time and improves stability, especially with many obstacle constraints.

The dynamics, cost, and constraints were implemented as separate MATLAB functions. The `drone2D-dynamics.m` calculates the time derivatives of the six state variables using the nonlinear equations presented earlier. It is used inside both the cost function and the constraint function. The `mpc-cost-2D.m` simulates the drone forward over the horizon and accumulates penalties on tracking error, control magnitude, and terminal deviation. Since the dynamics are included inside the cost, this function captures the full nonlinear behavior of the drone. The `mpc-constraints-2D.m` checks whether the predicted trajectory stays outside all obstacle boundaries. Each MPC step is subdivided and checked at three smaller intervals to ensure collisions cannot occur between discrete prediction nodes. This makes the collision checking more realistic and prevents the optimizer from slipping through obstacles numerically. This modular design made it easy to update individual components without breaking the entire simulation. All optimization problems were solved using MATLAB's FMINCON with the SQP-legacy algorithm. SQP handles nonlinear constraints smoothly and works well with differentiable cost functions, so it was well-suited for our scenario. Since the neighborhood map produced more than 50 obstacles, the number of constraints at each MPC iteration became quite large. This made the solver settings important. Tight constraint tolerances and reasonable limits on the number of iterations were used to keep the solver stable.

The MATLAB implementation links the nonlinear drone model, the quadratic MPC cost, and the obstacle-avoidance constraints into a working closed-loop system. The final structure provided realistic navigation simulations in a cluttered neighborhood environment and a flexible platform for testing different obstacle layouts, tuning parameters, and control strategies.

VII. Results and Discussion

A. 2D Path Planning

1) Trajectory Behaviour and Obstacle Avoidance: The generated 2D trajectory shown in figure 2 demonstrates MPC successfully avoiding the 53 obstacles and no-fly zone and reaching the target (460, 400) from the starting point, (0, 0). MPC horizon helps predict the obstacles, due to which the heading direction of the quadrotor is changed early to avoid collision with obstacles. The drone also chose the smooth corridor between the clusters of houses without violating the constraints. This shows that even with simple 2D dynamics, MPC can produce high-quality motion planning.

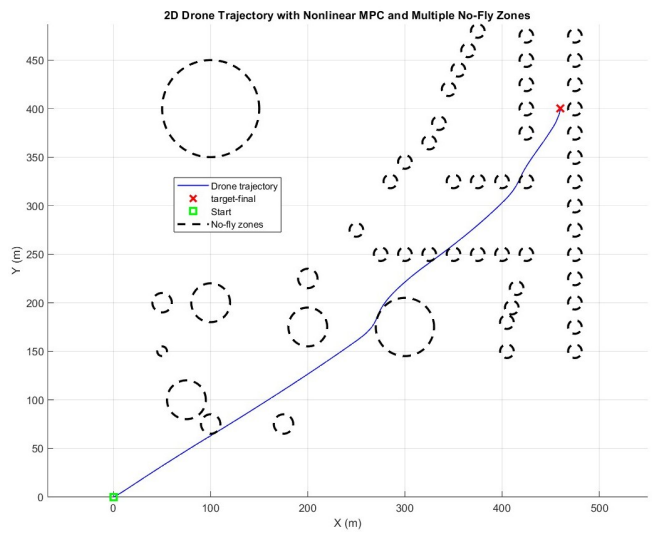

Fig. 2. Drone trajectory with nonlinear MPC.

2) Velocity Profile Analysis: The velocity evolution over time Figure 3 reveals how the drone adapts its speed based on obstacle size and density. The velocity in the x and y directions increases rapidly during the early phase as the path is relatively open. It is constant around 8.5 m/s, limited by damping and available acceleration. Around steps 160–200, sharp changes appear in both directions. This corresponds exactly to the region where the drone navigates around obstacles. The controller slows down to maintain safe clearance and reorients heading to avoid obstacles and to fit through a narrow corridor. It accelerates again once clear of obstacles. Near the target, both velocity components decrease smoothly to ensure accurate arrival. The velocity plot confirms that MPC does not simply minimize the cost and maximize the speed to reach the target but also modulates velocity intelligently based on dynamic constraints and available inputs.

3) Control Input Behavior: The evolution of the control effort by MPC for forward acceleration and yaw rate exhibits a smooth and nearly linear decrease over time. This is the result of penalizing the control efforts to minimize energy use. Thus, it gives us a smooth and linear decrease in the input applied. At first, a large input is applied to gain momentum

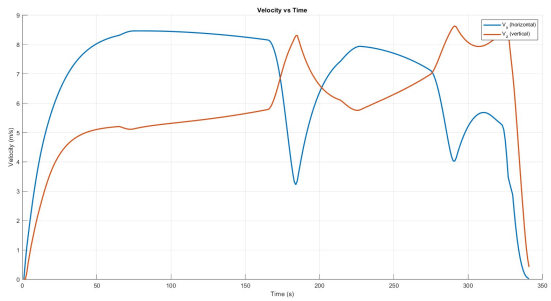

Fig. 3. Velocity Profile in X and Y direction

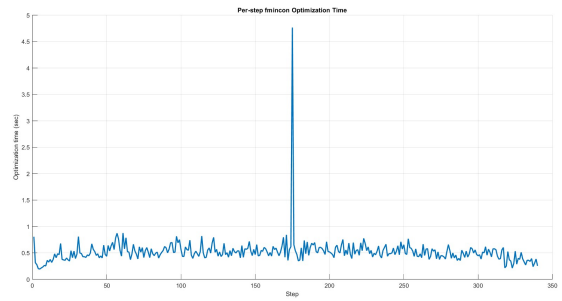

Fig. 6. Optimization time taken by the solver to optimize at each step

and align the heading angle appropriately. After that, control effort shifts from an initial high effort to minimal effort. The smooth decay in both control inputs indicates that the MPC avoids abrupt changes, minimizes unnecessary control effort, and maintains dynamic feasibility as shown in the Figure **??**

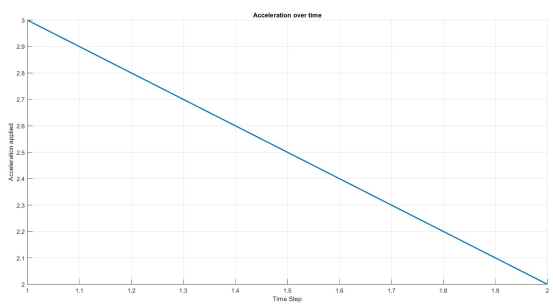

Fig. 4. Linear decrease in acceleration over time

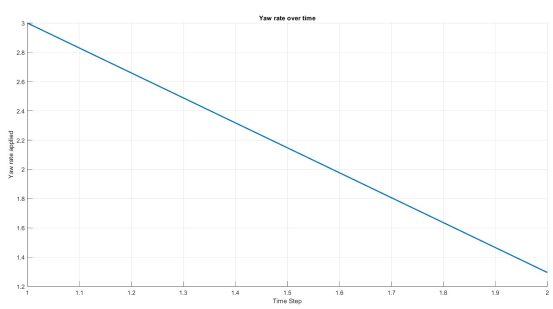

Fig. 5. Change in yaw rate over time

4) Optimization and Performance: The optimization time for each step is shown in the figure 6. For simulation, computation time remains between 0.2- 0.8 s for most of the time except at step 172. The reason for this is due to multiple obstacles in the direction of motion, and it requires more time to optimize and meet all the non-linear constraints. The optimization time for the SQP solver for non-linear dynamics and constraints is as expected. There are a total of 30 decision variables and more than 2300 non-linear inequality constraints for obstacles at each simulation step.

5) Energy Consumption: Figure 7 shows the evolution of the drone's remaining energy throughout the flight. The energy decreases almost linearly from the initial value of 100 units to approximately 69 units by the time the drone reaches the target. This behavior is consistent with the energy model explained in Section III for quadrotor dynamics. Since motion is almost smooth throughout the trajectory, and acceleration, a, and yaw rate, w, are also almost smooth. As a result, the energy decreases linearly rather than sharply. The reason for smooth input is to avoid the obstacles in dense regions, which is the desired result.

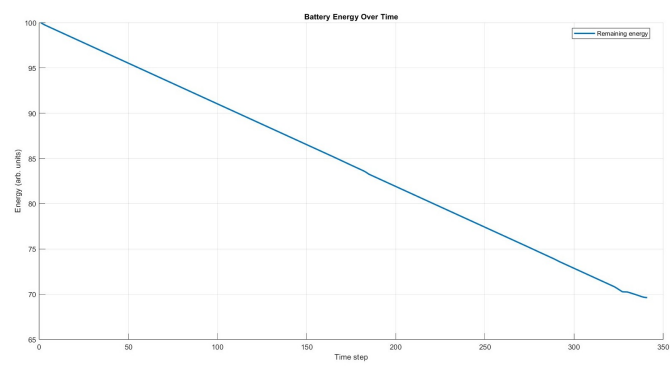

Fig. 7. Battery percentage evolution

B. 3D Path Planning

To explore the scalability of our nonlinear MPC formulation, we extended the 2D drone dynamics into 3D path planning problem with cylindrical no-fly zones 3D obstacles. Although this extension was developed within just two days, the controller successfully demonstrated the ability to guide the drone from its initial position to the 3D target while respecting the imposed obstacle constraints. As shown in Figures 8–9, the drone follows a feasible spatial trajectory that avoids the cylindrical obstacles while steering toward the destination. Additionally, the 3D obstacle arrangement used here is more basic, meaning finer testing, improved environment mapping, and more extensive validation would be required to get the desired results. This extension provides a strong foundation for future work, where a more detailed 3D

dynamics model, improved constraint handling, and real-time computational enhancements can be integrated.

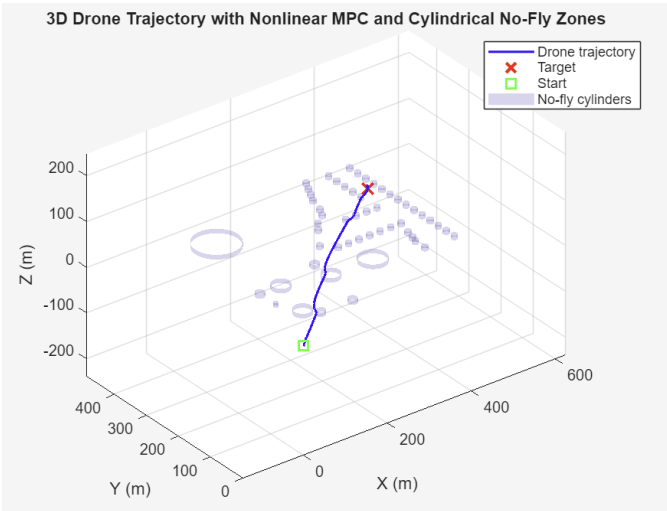

Fig. 8. Successful 3D path planning with obstacle avoidance

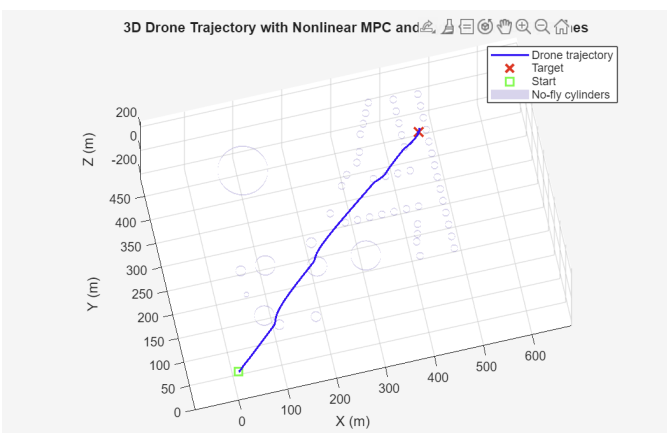

Fig. 9. Top view of the 3D path planning. Demonstrate successful navigation to the target.

VIII. Conclusion

This project demonstrated the effectiveness of nonlinear MPC for drone navigation in a complex, cluttered neighborhood environment. By combining a simplified but expressive nonlinear planar quadrotor model with quadratic penalties and explicit obstacle-avoidance constraints, the controller was able to generate smooth, collision-free trajectories around more than fifty no-fly zones. The use of FMINCON enabled the incorporation of nonlinear dynamics and constraints without requiring model linearization.

One key finding was the importance of terminal penalties and velocity weighting in the quadratic cost. These terms were essential for preventing overshoot near the target and for encouraging physically realistic deceleration. Similarly, the introduction of an inflated obstacle radius provided a practical safety buffer, ensuring that the drone did not graze building boundaries.

A. Future Work

Several extensions of the present work are promising for future study:

- **3D Full-Quadrotor Dynamics:** Extending the model to $\mathrm{SE}(3)$ would allow realistic altitude changes and more expressive maneuvers.
- **Real-Time Feasibility:** Implementing C/GMRES or PANOC-based NMPC would improve solve times for onboard embedded hardware.
- **Uncertainty Handling:** Incorporating chance-constrained MPC or tube-MPC would allow the system to handle wind disturbances and state-estimation noise.
- **Map-Based Planning:** Integrating global planning (e.g., RRT*) with local MPC can ensure both long-range optimality and short-range collision avoidance.

Overall, the project shows that nonlinear MPC is a robust and flexible approach for UAV navigation in real-world environments.

References

[1] D.-M. Lee, J.-H. Jung, Y.-S. Sim, and G.-W. Kim, “Tracking Control of Quadrotor Micro Aerial Vehicles Using Efficient Nonlinear Model Predictive Control with C/GMRES Optimization on Resource-Constrained Microcontrollers,” Electronics, vol. 14, no. 2775, 2025, doi: 10.3390/electronics14142775.

[2] J. C. Pereira, V. J. S. Leite, and G. V. Raffo, “Nonlinear Model Predictive Control on SE(3) for Quadrotor Aggressive Maneuvers,” J. Intell. Robot. Syst., vol. 101, p. 62, 2021, doi: 10.1007/s10846-021-01310-8.